

BetDoc

BetDoc is a high-frequency, algorithmic sports betting architecture built on strict Domain-Driven Design (DDD) principles. It is designed to act as an automated quantitative hedge fund for sports markets.

Architecture & Layers

BetDoc follows a deeply decoupled Hexagonal Architecture (Ports and Adapters). The core math and business logic never depend on external databases, web frameworks, or third-party APIs.

1. Domain (`src/betdoc/domain`)

The heart of the application. Contains pure, math-heavy business logic with zero external dependencies.

- **Math & Pricing:** EV Calculation (`ev_filter.py`), True Odds Devigging (`devig.py`), Asian Line Payouts (`asian_lines.py`).
- **Risk & Staking:** Robust Portfolio Kelly sizing (`portfolio_kelly.py`), Parlay Copula Correlation (`parlay_correlation.py`), Execution Risk modeling (`execution.py`).
- **Intelligence:** Domain models for the Twin Advisor (`advisor_models.py`), Account States (`account_models.py`), and Live News impact (`news_models.py`).

2. Application (`src/betdoc/application`)

The conductor of the system.

- **Event Bus:** Redis Streams wrapper (`event_bus.py`) that acts as the circulatory system.
- **Orchestrator:** (`orchestrator.py`) Consumes high-frequency tick data from the Event Bus, evaluates them through the Twin Advisor, and routes actionable alerts.
- **Ports:** Abstract Base Classes defining the contracts for Bookmakers, Account States, and Notifiers.
- **Resilience:** Circuit Breakers and retry mechanisms to protect against external API failures.

3. Adapters (`src/betdoc/adapters`)

Implementations of the Application Ports. This is where the system talks to the outside world.

- **Bookmakers:** Implementations like `TheOddsApi` for fetching live odds.
- **Cache & Persistence:** `InMemoryStateStore` (L1 Cache) and Ledger/DB modules.
- **Notifiers:** `RichConsoleNotifier` for Bloomberg-terminal style dashboards with a built-in `AntiSpamRegistry` (Debouncer).

4. Presentation (`src/betdoc/presentation`)

The invisible API Gateway that bridges the command-line engine to web frontends.

- **FastAPI App:** Runs an embedded Uvicorn server as a background task.
- **REST API:** Exposes endpoints to fetch `AccountState` and Realized PNL (`/api/v1/intelligence/accounts`).
- **WebSockets:** Streams real-time `TwinRecommendation` alerts to connected dashboards with dedicated async queues per client to prevent backpressure blocking.

5. Services (`src/betdoc/services`)

High-level service classes.

- **Twin Engine:** The core `TwinAdvisorService` (`advisor/twin_engine.py`) that marries the risk profile to the EV filter to the Portfolio Kelly sizer, acting as the ultimate gatekeeper for all capital allocation.

Tech Stack

- **Python 3.12+**
- **Pydantic V2:** For ultra-fast validation and serialization.
- **Asyncio / FastAPI:** For non-blocking, high-concurrency event loops.
- **Redis:** For Event Streams (Tick ingestion) and Pub/Sub.
- **Rich:** For terminal UI.
- **CVXPY & NumPy:** For complex portfolio optimization and correlation copulas.

How to Run

Ensure Redis is running locally or configured via `.env`.

```
# Install dependencies
pip install fastapi uvicorn websockets pydantic pydantic-settings redis structlog rich numpy scipy cvxpy

# Run the full Orchestrator & API
python src/betdoc/main.py
```

BetDoc: System Architecture & Tech Stack

1. System Overview

BetDoc is an **Event-Driven, High-Frequency Quantitative Betting Engine**. It is designed to ingest live sports odds from multiple sources in real-time, store them in ultra-fast memory, and run complex mathematical models (Arbitrage, +EV, Kelly Criterion) to identify market inefficiencies before bookmakers can correct them.

2. The Tech Stack (Gold-Grade)

Core Backend

- **Language:** Python 3.11+
 - *Why:* The industry standard for quantitative finance and machine learning. We enforce strict typing (`mypy`) and 100% asynchronous execution (`asyncio`) to prevent bottlenecking.
- **Framework:** FastAPI
 - *Why:* Lightning-fast backend framework. It natively supports asynchronous endpoints and WebSockets, which is mandatory for pushing live alerts to a dashboard.

Database & State Management

- **Hot Storage (The Live Brain):** Redis (`redis.asyncio`)

- *Why*: When a match is live, odds change every second. Standard databases (like SQL) are too slow to read/write this fast. Redis stores the live odds directly in your computer's RAM, allowing the math engine to access them in sub-milliseconds.
- **Cold Storage (The Archive)**: PostgreSQL + SQLAlchemy
 - *Why*: Used for long-term storage. This tracks our betting history, bankroll growth, and Closing Line Value (CLV) to prove our mathematical edge over months of data.

The Math & Data Layer

- **Libraries**: `numpy`, `pandas`, `itertools`
 - *Why*: We use vectorized math operations to cross-reference thousands of odds combinations instantly without slowing down the server.
-

3. The Architecture (How Data Flows)

The system is broken into strictly separated "Engines" so that if one piece breaks, the whole system doesn't crash.

1. **Data Sources / APIs** stream data into the system.
2. **Ingestion Engine** (`base_api.py`) normalizes the messy data into a clean, standard format.
3. **Redis (Hot State)** catches the clean data and stores it in RAM.
4. **Quant Engine** constantly listens to Redis. The microsecond new odds arrive, it runs:
 - *De-Vig Math*: Removes the bookmaker's margin to find the True Probability.
 - *+EV Scanner*: Compares True Probability against soft bookmakers to find profitable gaps.
 - *Kelly Criterion*: Calculates exactly what percentage of the bankroll to risk.
5. **FastAPI Server** (`main.py`) takes the profit signals and pushes them to the user via WebSockets, while simultaneously saving the record to **PostgreSQL** for historical tracking.

